FINAL PROJECT

2026-06-19

AUTOMATIC ANALYSIS AND GRADING OF UTML UML DIAGRAMS

Douwe Osinga

d.r.osinga@student.utwente.nl

Supervisor
dr. ir. Vadim Zaytsev

v.zaytsev@utwente.nl

Supervisor
dr. Nacir Bouali

n.bouali@utwente.nl

Faculty of Electrical Engineering,
Mathematics, and Computer Science

UNIVERSITY OF TWENTE.

ABSTRACT

During computer science studies, students are often required to submit diagrams. The grading of these diagrams is currently done by humans, resulting in a costly, lengthy, and error-prone process. In this paper, we investigate the theoretical feasibility of automatically grading diagrams, focusing on Unified Modelling Language diagrams and the UTML software and file format used by the University of Twente. Existing work shows that graph isomorphism algorithms which incorporate algorithms to account for the use of synonyms and the presence of spelling mistakes provide the best grading results. However, autograders utilising these techniques are not reproducible or open-source. Therefore, we propose *Seshat*, an open-source autograder that combines the aforementioned techniques and is capable of supporting arbitrary diagrams and file formats, with built-in support for UTML. comparison to human grading here

1. MOTIVATION

Unified Modelling Language (UML) diagrams, introduced by the Object Management Group [1], play a significant role in computer science, as they allow for communicating software designs in a standardised format. During technical studies, students are often required to make such diagrams for graded assignments or exams.

However, the grading of these diagrams is often a costly and lengthy process, involving multiple paid staff members [2][1]. Additionally, this process is prone to grading inconsistencies due to various reasons [2], with a major factor being the inherent inconsistency of human graders [2] [3]. M. Meadows *et al.* [3] pose two possible solutions to the problem of human grading: either "report the level of reliability associated with marks/grades, or find alternatives to [grading]." We propose a third alternative: finding alternatives to the grading *process*.

The (partial) automatisation of grading diagrams ('autograding') a grading paradigm that can both reduce the cost and time required for institutions and reduce the inherently present inconsistencies in human grading[2] [4] [5]. This could result in similar or superior performance compared to human grading in terms of **accuracy**, **grading transparency**, and **consistency**.

With *accuracy*, we mean the percentage of points assigned to a submission that are prescribed by the rubric for a particular excercise. With *consistency*, we mean both the extent to which similar grades are given to similar submissions and the difference between consecutive runs (i.e. determinism). With *grading transparency*, we mean the extent to which the reasoning for a particular grade is explained with regards to the rubric for the exercise or to the Intended Learning Objectives (ILOs) of a module. These properties are desirable in the grading process, as it means that students are graded in a way that reflects their performance (*accuracy*), allows them to see which parts they could improve for future assignments (*grading transparency*), and is minimally unfair (*consistency*).

Specifically for the implementation, we desire an autograder that can *link its grading to ILOs*, as described by the previous paragraph. Furthermore, built-in *UTML support* is a must, as it is the main file format the University of Twente uses. Finally, an *easily extensible* autograder would be ideal, since we might want to extend support to other file formats or alter the behaviour of the autograder later on.

1.1. A brief history of autograding

The idea of letting a computer program (partially) grade tests has been discussed in papers since the 70s [6, p.13], with some documented implementations starting to appear around the 80s. These were primarily focused on grading the writing style of computer programs [7]. Interest in grading diagrams specifically seems to have started around the early 2000s [8] [9].

1.2. Diagrams categories and formats

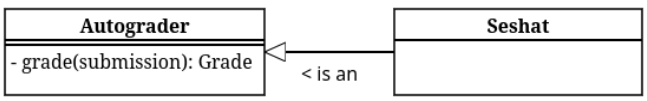

Figure 1: An example UTML UML class diagram.

Diagrams themselves have several categories, each for different purposes. UML diagrams, as shown in Figure 1, mainly serve to visualise and

[1] From personal experience.

document software [1], while Entity-Relation diagrams, such as shown in Figure 2, focus on the relations between different components, making it ideal for visualising database designs [10].

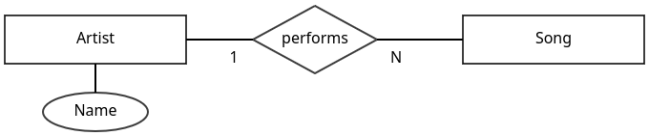

Figure 2: An example Entity Relation diagram in the Chen format.

Different formats exist for storing these diagrams. Examples include XMI - the standard diagram interchange format for UML, most commonly used by the Eclipse Modelling Framework [11], the Rose Petal format - used by IBM Rational Rose [12], PlantUML - an open-source textual standard for representing various diagrams including UML and ER diagrams [13], VPP files - used by Visual Paradigm, software that allows for modelling UML, architecture diagrams, business flows etc. [14], and UTML - a program and file format used at the University of Twente for representing UML and various other types of diagrams, its file format building on the JSON standard [15] [16].

The degree to which automated grading is implemented can vary as well. We divide autograding into the following categories: *non-automated* (everything must be done by a human), *automated* (part of the process requires no human input), and (fully) *automatic* (no human input is required). In this paper, we only consider autograders that fall into the categories *automated* and *automatic*.

2. RESEARCH QUESTIONS

In order to examine the feasibility of automatically grading UML diagrams saved in the UTML format, we provide a main research question (**MRQ**):

To what extent can UML diagrams be graded automatically while maintaining or improving the accuracy, consistency, and grading transparency of human grading?

We aim to answer the main research question with the following research questions:

RQ1: To what extent are existing solutions suitable for use in autograding UTML diagrams with regards to (1) accuracy, (2) consistency, (3) grading transparency, (4) extent of linking ILOs to grading instructions, (5) UTML support, and (6) extensibility?

RQ2: To what extent can a suitable autograder be constructed from previous work to be able to grade UTML diagrams?

RQ3: To what extent does the suitable autograder compare to human grading in the context of grading first-year UML exam questions?

RQ1 is answered in Section 3, by analysing existing work for suitability of grading. These works are categorised according to the requirements outlined by **RQ1** and presented in Table 1. Section 4 explains the features and reasoning behind *Seshat*. Section 6 offers perspectives into its performance compared to human grading, after which we discuss and conclude in SECTION TODO.

3. RELATED WORK

In order to answer research questions **RQ1**, we conduct a small-scale literature mapping study [17]. We aim to provide an comprehensive, but not exhaustive, view into the world of autograders and ILOs, which is why we omit formal inclusion and exclusion criteria. Works are collected using the search engines Google Scholar[3] and ResearchGate[4]. Autograder material is queried with terms including but not limited to "automatically grading UML diagrams", "autograder diagram", "UML diagram assessment", "machine learning diagrams", and "diagram evaluation assessment AI". Material regarding ILOs and ILO integration into rubrics was searched with terms such as "learning outcomes include in rubric", "learning objectives in rubrics", and similar. Snowballing (the practice of looking at sources of sources) is used, to a depth of 1. Af-

[2] Given that the process is deterministic.
[3] https://scholar.google.com
[4] https://www.researchgate.net

ter starting with roughly 40-45 works and removing the non-relevant or less noteworthy papers, we arrive at a compact set of sources that offer a general view of autograding across literature.

3.1. Autograders

Multiple technologies and strategies for autograding exist, which we categorise into the following categories: frameworks for autograders, purely algorithmic autograder implementations, and Generative AI (GenAI) or Machine Learning (ML) implementations. Findings on implementations are summarised in Table 1.

3.1.1. Frameworks / Theoretical

Autograder frameworks dictate certain designs or methodologies for building autograders. We present summaries of the most relevant explored frameworks, and provide a general summary at the end.

N. Smith *et al.* [8] provide a five-step framework for assessing "possibly ill-formed or inaccurate diagrams" that include the steps (1) segmentation, (2) assimilation, (3) identification, (4) aggregation, and (5) interpretation. While the first two steps are meant for translating images or other "raster-based input" into diagrammatic primitives, which is not useful for us, the latter stages provide a solid conceptual foundation to grade diagrams.

F. Batmaz [18] takes a broader look at the grading process, identifying and developing techniques to reduce repetitive actions, focusing on database ER diagrams. The paper suggests a semi-automatic grading system, including automatic grading based on identifying identical segments between a submission and the solution.

V. Vachharajani *et al.* [19] propose an architecture specifically for UML use case diagrams, providing a useful catalogue about edge cases related to diagram creation, such as the chance of misspellings, synonyms, abbreviations, directionality of relationships, to list a few.

W. Bian *et al.* [20] establish two models: one to map submissions to example solutions and one to grade submissions. It recommends syntactic matching to help with spelling mistakes, semantic matching to match related words, and structural matching to match neighbouring elements and/or inheritance, with the goal to most efficiently match parts of a student submission with a sample solution.

In conclusion, most autograder frameworks recommend the same set of techniques for autograding: structural matching to identify similar segments of graphs, often in combination with syntactic matching that accounts for misspellings and semantic matching to account for synonyms or related words.

3.1.2. Algorithmic

Some papers also mention concrete implementations of autograders, some of which based on the aforementioned frameworks, of which a subset uses purely algorithmic methods. Summaries of these sources are discussed in this section, along with a general summary on algorithmic autograders.

W. Bian *et al.* [5] implement their previously mentioned framework [20] and validate it using a case study. They use the Levenshtein distance between words for syntactic matching, several algorithmic metrics for semantic similarity, and structural matching based on similar attributes and operations within classes. They find that comparing submissions to multiple solution variants results in more accurate grades with an average accuracy over 95% [5, p.10]. Additionally, they find that grading configurations need to change per exam if the goal is to produce the most similar scores to manual grading, likely because the focus of each exam or exercise lays on a different aspect of diagram creation. Additionally, they state that autograding "has shown to be more consistent and able to ensure fairness in the grading process" compared to manual grading [5, p.11]. Finally, their visual feedback system seems to be a useful addition, clearly visualise grading results which is likely to be more intuitive for both graders and students compared to pure text output. An example is shown in Figure 3.

M. Hosseinibaghdadabadi *et al.* [21] also implement W. Bian *et al.* [20]'s framework, comparing UML use case diagrams to one or multiple example solutions and preferring the maximum grade. They use a graph similarity algorithm

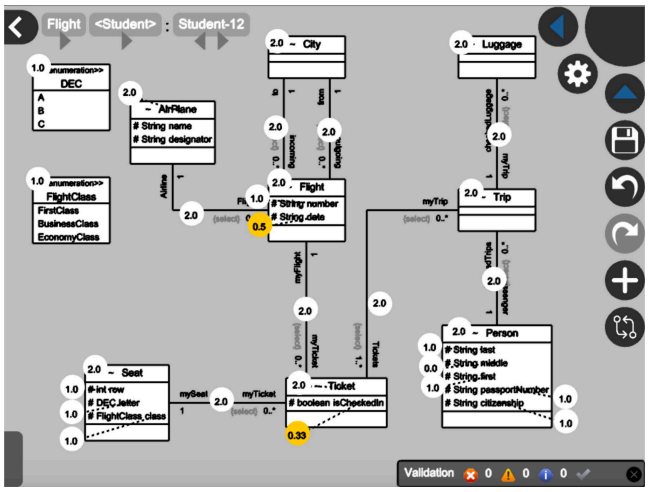

Figure 3: Visual feedback module from W. Bian *et al.* [5, Fig.9]

which matches nodes based on structural matching, along with syntactic matching using the Levenshtein distance between words and semantic matching using a WordNet similarity score (HSO, WUP, LIN metrics). It achieves a similarity percentage of 93.31% [21, p.114].

O. Anas *et al.* [22] compare UML class diagram submissions to an example solution. They use a graph similarity scoring algorithm based on structural matching along with syntactic and semantic matching. Syntactic matching is done with substring matching and semantic matching is done with neighbour similarity ("the comparison of the neighboring classes" [22, p.1585]), relationship name, type, multiplicity, and inheritance. It achieves a respectable correlation with human grading, with more than 80% of submissions receiving identical grades, more than 90% of grades with a corelation larger than 0.85, and no grades with a correlation lower than 70%.

H. AlRawashdeh *et al.* [23] provide an interesting alternative way of grading submissions: they combine many UML diagram validators, model checkers, and even use LTL properties. However, a clear purpose, scope, and results are missing from the paper.

M. Striewe *et al.* [24] also attemp property checking akin to the work of H. AlRawashdeh *et al.* [23] by focusing on graph queries for evaluation, providing a domain-specific language that looks similar to SQL. While it offers an interesting alternative approach, the fact that teachers would have to learn a query language and transform their existing rubrics/example solutions into this format could be a real hurdle, especially given the performance of graph-isomorphism-based solutions [5] [21] [22]. Additionally, the paper does not account for misspellings or the use of synonyms.

P. Thomas provide a selection of works detailing the autograding of ER diagrams which account in their basis for diagrams containing misspellings, duplicate entities, and other imprecisions [9] [25] [26] [27] [28]. They analyse graphs by comparing ever increasing subgraphs called (Minimal) Meaningful Units, based on the work of N. Smith *et al.* [8]. By 2009, P. Thomas *et al.* manage to achieve a correlation to human grading of 92%, along with statistically proving that their autograder grades more consistently than human grading. The correlation to human grading can be seen in Figure 4.

In 2011, P. Thomas *et al.* also mention an online platform for both students and teachers to ease the process of automatic grading further, also used by N. Smith *et al.* [29], which additionally contains more mathematical explanations of their previous work. Unfortunately, the source code of this autograder is untraceable.

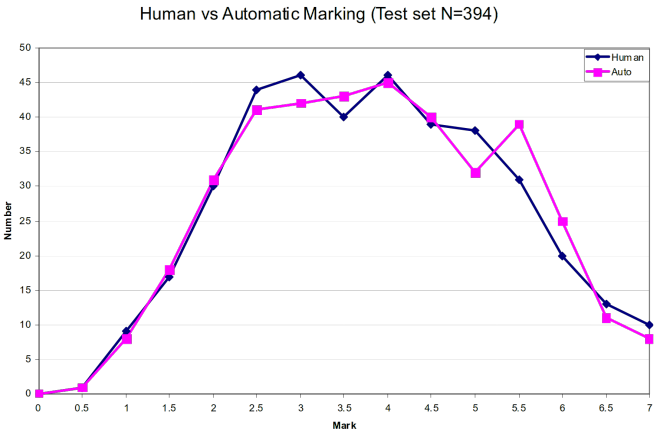

Figure 4: P. Thomas *et al.* [27, Fig. 3]: Human vs. automatic grading in database ER diagrams.

In conclusion, most existing implementations of autograders use some graph isomorphism algorithm with a combination of structural, semantic, and syntactic matching, as suggested by the majority of discovered frameworks. Some solutions attempt to autograde using property or formula checking, but fail to mention a detailed enough methodology or results to warrant further investigation.

3.1.3. Machine Learning / Generative AI

Next to using purely algorithmic methods, some papers experiment with Machine Learning / Generative AI (ML/GenAI) to automatically grade submissions. There are even some hybrid ML and algorithmic solutions. We provide summaries of the explored sources below, along with a general conclusion on ML/GenAI autograders.

D. R. Stikkolorum *et al.* [30], one of the earliest found sources, attempts Machine Learning-based autograding using several machine learning algorithms to compare submissions to expert grades. Unfortunately, the grading only reaches a maximum accuracy of 42.76%, while rounding off scores to a 10-point integer scale. Exact methods and algorithms are not mentioned.

C. Wang *et al.* [31] evaluate the feasibility of LLM-based grading with the LLM model ChatGPT-4o, focusing on student reports containing multiple types of UML diagrams. They feed pictures of student-submitted UML diagrams directly into the model along with an explanatory prompt that aims to trigger a Chain-of-Thought process (which should help LLMs "tackle complex arithmetic, commonsense, and symbolic reasoning tasks" [32]), and runs the model once per submission, with a temperature of 0.1. They find that score differences range from −0.25 to +3.75 points, with with the LLM handing out significantly lower average scores compared to humans. Additionally, they note many occurrences of incorrect grading (wrong identifications, overstrictness, and misunderstandings) [31, p.18], which means that, while the authors claim that their solution "demonstrates particular proficiency in the automated evaluation of UML use case diagrams", the grading is not internally consistent and contains hallucinations. In the words of the authors: "In the evaluation based on UC4, GPT deducts points for missing relationships between specified actors and use cases, but theses relationships existed in the UML use case" [31, p.13]. Furthermore, the paper does not express a strong correlation between LLM grading and human grading, at least compared to papers utilising graph matching algorithms [27] [21], nor does it recognise the inherent bias of LLMs [33] or their inherent nondeterminisim (even with a zeroed temperature) [34] [35].

N. Bouali *et al.* [36] uses various LLMs (Llama 3.2B, ChatGPT-o1 mini, and Claude Sonnet) to grade diagrams, first translating the diagrams into text instead of giving the LLM images directly like C. Wang *et al.* [31]. While they achieve a Pearson correlation to human grading of 0.76 with both ChatGPT and Claude, they run into the same inconsistency issues as C. Wang *et al.*: "while the models would provide a final score as requested in the prompt's response format, this score often did not match the actual sum of points awarded in their criterion-by-criterion assessment", and "'One ChargingPort is associated with One Vehicle' was matched with 'One ChargingPort is associated with One ChargingStation' with a similarity of 0.92, despite describing different domain relationships" [36, p.164].

N. Bouali *et al.* identify the problem with grading with LLMs perfectly, stating that "This discrepancy can be attributed to the autoregressive nature of LLMs, where they generate responses token by token" [36, p.164]. Because these models are in their very essence based on predicting tokens [37], there is no formal guarantee that the grades are internally consistent and that grades are produced accurately with respect to the rubric. The fact that LLMs produce grades that correlate with human grading does not mean that this grading is done in a fair, consistent, or reliable manner. While N. Bouali *et al.* try to reduce the nondeterminisim of LLMs by setting the temperature to zero, this does not necessarily remove non-determinism [34] [35], nor does it account for training biases [33], as mentioned before.

R. Ramachandran *et al.* [38], unlike the previous papers, use a human-in-the-loop design in combination with both purely algorithmic steps, using LLMs only for semantic and syntactic matching. Using structural matching algorithms similar to papers presented in Section 3.1.2, it achieves a Mean Average Error of only 0.611, aligning very closely to human grading (see Figure 5). Unfortunately, the data set contains only ten self-procured diagrams, which negatively impacts the significance of these results,

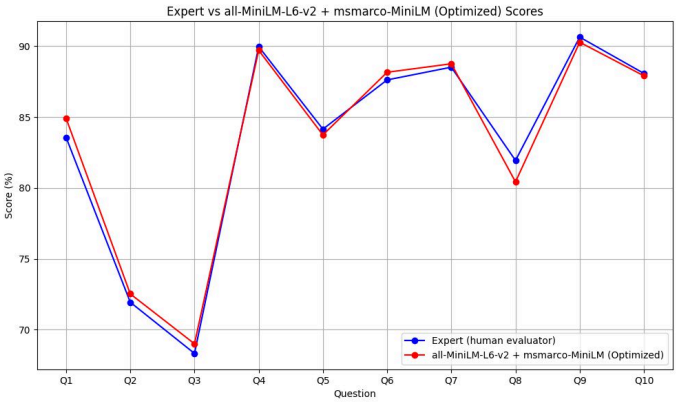

Figure 5: R. Ramachandran *et al.* [38, p.13]: Comparison of expert scores and CodeLLama scores using a combination of `all-MiniLM-L6-v2` and `msmarc-MiniLM` as word similarity models.

not to mention that the nondeterminism introduced by the LLMs will impact the consistency of grading, although it is unclear to what extent.

In conclusion, while ML/GenAI grading has been attempted in recent years, purely ML/GenAI solutions produce lacking similarity to human grading compared to graph isomorphism-based solutions. Additionally, GenAI solutions introduce non-deterministic and biased behaviour, while providing no consistency guarantees. This makes using only these types of solutions inferior to graph isomorphism solutions in terms of *consistency* and *grading transparency*. When used only for semantic matching, however, it can provide equal or possibly superior accuracy to algorithmic solutions, although one must be careful to not introduce nondetermism in the grading process this way which would negatively affect *consistency*.

3.2. Intended Learning Objectives and examination

A. Dinur *et al.* [39] states that analytical rubrics (those which mention explicit criteria) provide more details than global, holistic rubrics. These types of rubrics (and their exercises) can be constructed directly from the ILOs of a course or module, which would provide a detailed grading rubric that aligns closely to its ILOs [4]. Given that rubrics and exercises are defined in such a way, one could link these ILOs to the grading rubric and provide functionality for an autograder to show these in the final grade This would indicate to students how well they achieved the learning goals of the module in order to improve *grading transparency*.

3.3. Conclusion

In the explored related work, existing frameworks primarily recommend structural matching in combination with syntactic and semantic matching to account for spelling mistakes and the use of synonyms. Existing implementations mostly use the methods recommended by the frameworks, with the best results stemming from determinstic graph isomorphism algorithms. Purely AI-driven methods may require less effort from teachers, since teachers do not need to produce sample diagrams but can describe their rubric in words, but produce noticeably inferior results to graph matching algorithms. Using hybrid methods, specifically using AI-driven classification algorithms only for semantic/syntactic matching, seems to produce similar results to 'pure' graph matching algorithms, but does not necessarily provide accuracy gains over algorithmic solutions and can additionally introduce nondeterminism in otherwise determinstic solutions, reducing consistency.

4. SESHAT

In order to automatically grade student submissions, we develop *Seshat* [48]: a generic autograder capable of automatically analysing and grading theoretically any type of diagram. For the purposes of this paper, we offer built-in support for UTML.

4.1. Techniques

Seshat uses the techniques from Section 3 and Table 1 which gave the best results in terms of accuracy, consistency, and grading transparency: a graph isomorphism algorithm for structural matching, Levenshtein distance for syntatic matching, and `all-MiniLM-L6-v2` [49] for semantic matching.

We first tried out Princeton's WordNet [50] as semantic matching, as it also performs semantic similarity checks and was mentioned in multiple papers [20] [21], but these scores did not reflect the expected semantic similarity after testing

[5] See `semantic_match_test.go` [48]

Met.	Author	Diagrams types	Ac	Co	Tr	F	A	I	R	ILO	UTML
Alg	W. Bian *et al.* [5]	UML Class	H	H	H	-	-	i	r	N	N
Alg	M. Hosseinibaghdadabadi *et al.* [21]	UML Use Case	H	H	H	-	-	i	r	N	N
Alg	O. Anas *et al.* [22]	UML Class	M	H	H	-	-	i	r	N	N
Alg	S. Modi *et al.* [40]	UML Class	?	H	H	-	-	-	-	N	N
Alg	R. Jebli *et al.* [41]	UML Class	?	H	H	-	-	-	-	N	N
Alg	N. H. Ali *et al.* [42] [43]	UML Class	?	H	L	-	-	-	-	N	N
Alg	H. AlRawashdeh *et al.* [23]	UML State/Sequence	?	H	?	-	-	I	-	N	N
Alg	M. Striewe *et al.* [24]	UML Class	?	H	H	-	-	I	-	N	N
Alg	S. Foss *et al.* [44] [45] [46]	ER	?	H	?	-	-	I	r	N	N
Alg	P. Thomas *et al.* [9] [25] [26] [27] [28]	ER	H	H	H	-	-	I	r	N	N
ML	D. R. Stikkolorum *et al.* [30]	UML Class	L	L	L	-	-	-	-	N	N
GenAI	C. Wang *et al.* [31]	UML	M	L	M	F	a	I	r	N	N
GenAI	N. Bouali *et al.* [36]	UML Class	M	L	M	F	a	I	r	N	N
Alg/ML	R. Ramachandran *et al.* [38]	ER	H	H	H	F	a	i	r	N	N

Table 1: Autograders and their suitability scores.

Explanation of columns: *What* **Met***hod is used* (Alg(orithmic), ML, GenAI), which **Diagram types** *are supported*, *how high is the* **Ac**(*curacy*), **Co**(*nsistency*), and *Grading* **Tr**(*ansparency*), *how* **F***indable,* **A***ccessible,* **I***nteroperable, and* **R***eproduable is the tool*, *can the tool link* **ILO***s to grading*, *and how well is* **UTML** *supported*?

Scoring is generally divided into "N" (*No Support*), "L" (*Low*), "M" (*Medium*), "H" (*High*), and "?" (*Unknown*), which gives an indication of each autograder's suitability w.r.t. that particular criterium. The scoring for these rubrics is done in a comparative way, with the lowest-scoring solution receiving a "L" or "N" and the highest scoring receiving a "H". High **accuracy** is awarded for deterministic solutions, with lower values given to nondeterministic programs. High **consistency** is awarded for determinstic solutions. High **grading transparency** is awarded for solutions that explain the final grade in terms of rubrics (medium for full rubrics that might not match (i.e. GenAI solutions)). **ILO** and **UTML** support is given a "H" or "N" based on inclusion of these features.
FAIR scoring is done by verifying the Findability, Accessibility, Interoperability, and Reusability, inspired by M. D. Wilkinson [47, Box 2, p.4]. We specifically look at the autogarder program. For example: if the code is findable with permalink, the project is available but only through a paywall, the algorithms in the paper are not designed to be interoperable with other diagram formats or types, and the work contains partial algorithms for autograding, it gets a score of 'Fa_r'.

with several self-synthesised examples and some example synonyms from the datasets[5]. This is likely the case because WordNet strictly matches according to the hierarchy of singular words, meaning that examples in the dataset such as 'ChargingPort' v.s. 'ChargingStation' would not be matched since 'Port' is in a different WordNet category than 'Station', even though in the exercise they are semantically similar, while examples such as 'Team Member' and 'Virtual Machine' got a semantic match, while sharing no semantic similarity within any of the datasets. It may of course also be the case that our comparison algorithm, which splits both the reference and submission text into words, compares all reference words to all submission words, and adds up all similarities, is not the correct way to approach this manner.

4.2. Architecture and Language

The goal of *Seshat* is to take input (from either exam exports, a list of files, or via some other format), transform the input into an internal graph representation, run comparison algorithms on it defined in Section 3.1.2 which produces a set of scores (a 'grade'), and format this grade in a certain way. The exact methodology, algorithms, and visualisations are likely to change, which is why we aim to maximally decouple these parts.

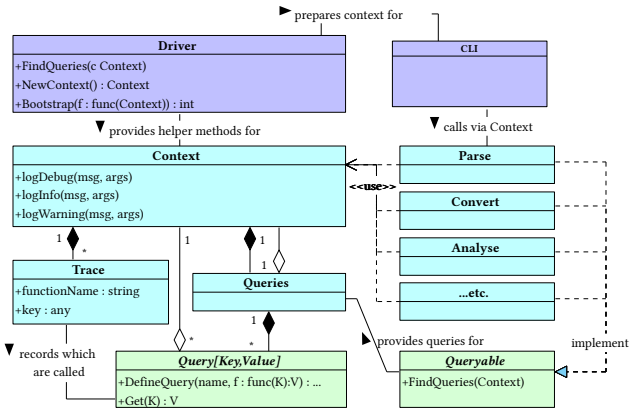

Figure 6: Query architecture for *Seshat*.

In order to achieve this, we implement a query-based architecture akin to that of the Rust compiler [51] (an example is given in Figure 6). This encourages decoupling each stage of the process, and additionally increases transparency

internally in the grading process, as one can easily query intermediate solutions from the grading process. Testing components is also inherently made easier due to the split-up functionality.

This architecture also allows us to cache all stages of the grading process if they are split up into separate queries, which should allow for some improvements in grading speed. Note that this is only possible because the autograder only contains determinstic algorithms.

The autograder is entirely written in Go [52]. We opted for Go as it is quite fast, strict enough to enforce the architecture seen in Figure 6, but still has a garbage collector which speeds up development and eliminates memory leaks. Additionally, a Command Line Interface (CLI) is included to showcase an implementation that works with the query architecture, as well as provide the author with an easy way to grade the datasets.

4.3. Features

This section describes the feature set of *Seshat* by walking the reader through the process of grading a dataset. Features are explained in the order of the grading process.

4.3.1. Parsing

The parsing step transforms a diagram file into an structure that *Seshat* can understand. For UTML, it merely parses the JSON UTML structure and adds some metadata such as the file name.

4.3.2. Transformation into internal representation

In order to be able to grade diagrams, *Seshat* uses an internal representation of a graph. This choice is made to separate the grading from any one specific diagram format, a mistake made in some other autograders (see Table 1). Separating the grading from a diagram format has the benefit of being able to easily integrate new diagram formats into *Seshat* by merely writing a translation from a particular format into an internal representation.

To support as many diagram formats as possible, this internal graph definition is very loosely defined. It contains as little semantic concepts as possible (inheritance, multiplicities, etc.) as it aims to capture the literal shapes and text. This allows *Seshat* to store possibly broken submissions, which allows us to explicitly check the well-formedness of a graph at a defined stage, or ignore certain broken aspects of graphs, which helps in the perceived *leniency* of the autograder and should bring it closer to human grading.

The structure is defined by some metadata such as the filename and a collection of vertices and edges. Each vertex and edge has a unique identifier. Vertices additionally contain a title, some values (fields or methods), certain optional semantic properties such as visibility (public/private/protected/…) and type (Class/Interface/…), and visual properties such as location and size. Edges are always directed and can connect at either end to a vertex, edge or nothing. Next to an identifier, they have stylistic properties for the starting and ending point of the edge, such as arrow style and text, as well as general stylistic properties, such as the style of the edge (dotted or solid). Finally, each edge has visual properties that define the edge's path. This path can be of any length, allowing for complex paths.

4.3.3. Error correction

After a diagram is converted into *Seshat*'s internal representation, *Seshat* tries to correct specific kinds of mistakes made in the diagram creation process, depending on which options are enabled and how they are configured. This also allows for encoding semantic meaning into the diagram if the original diagram format does not allow for expressing certain semantics.

For example, since UTML does not allow edge-to-edge connections, the 'reparation' phase can correct for this, given that edges are sufficiently visually close. These error-correcting and/or semantic encoding features exist to allow maximum leniency in grading and exist on the internal representation level, meaning they automatically apply to all diagram formats *Seshat* supports.

For visualising error corrections, we refer to submission 154286, shown in Figure 7, as well as its corrected version, shown in Figure 8.

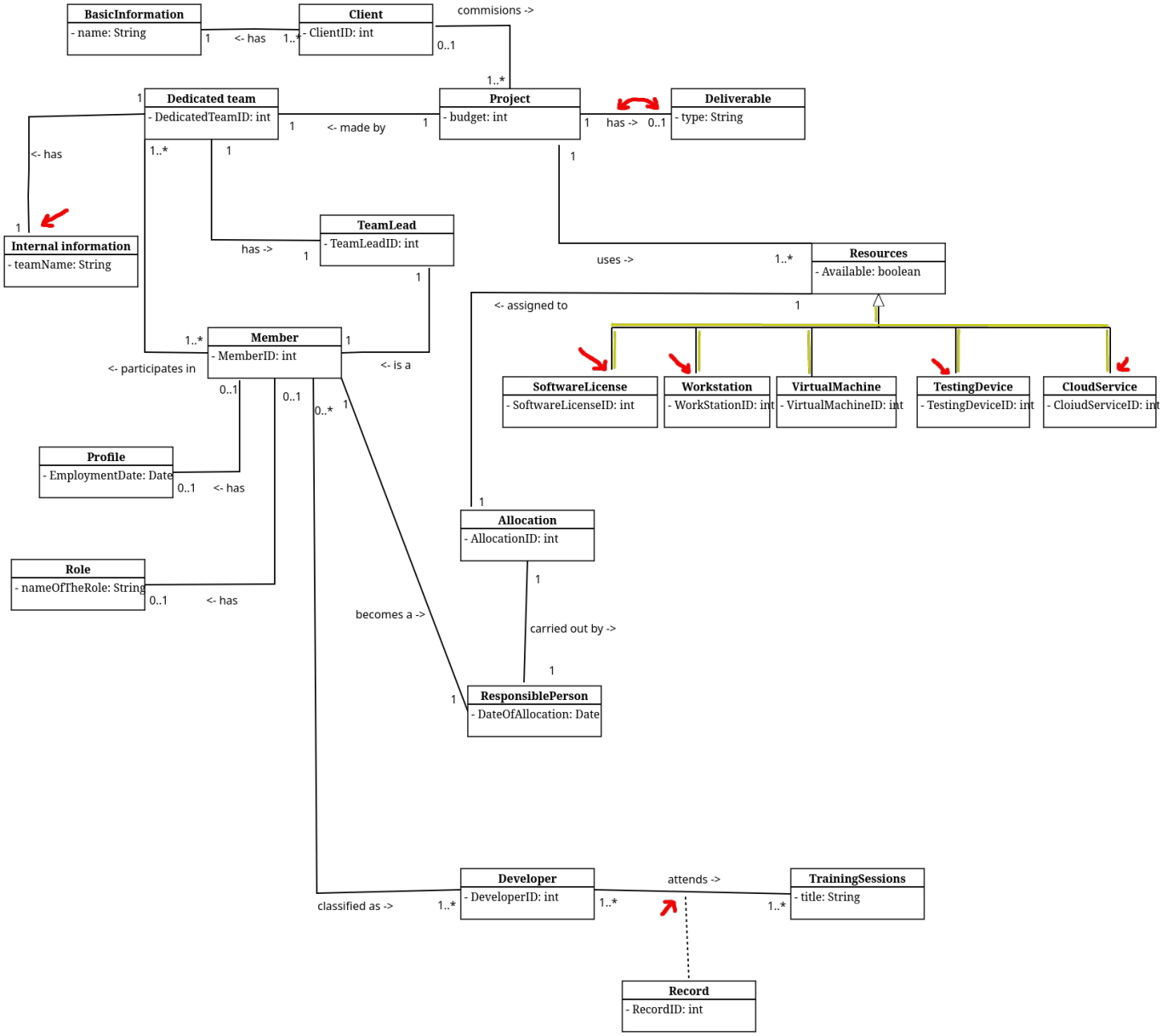

Figure 7: A student submission [48, 2025_M2_BIT/q/1/154286.json] containing several inconsistencies (marked red and yellow): 'Internal Information' has a disconnected edge. The edge between 'Project' and 'Deliverable' has internally swapped labels (see Listing 1). The edge of 'Record' is not connected to the 'attends ->' edge between 'Developer' and 'TrainingSession'. The inherited classes of 'Resources' are all separate edges, and most edges are not connected to the inheriting classes.

```
...
"middleLabel": { // middleLabel denotes the center of the edge
  "offset": { //...but the offset places it at the location of 'endLabel'
    "x": 45,
    "y": -6
  },
  "edgeLocation": 1,
  "value": "0..1"
},
  "endLabel": { // endLabel denotes the end of the edge
    "offset": { //...but the offset places it at the location of 'middleLabel'
      "x": -70,
      "y": -6
    },
    "edgeLocation": 2,
    "value": "has ->"
  },
  ...
```

Listing 1: 154286.json lines 96-111, showing internally flipped labels. From the graph of Figure 7.

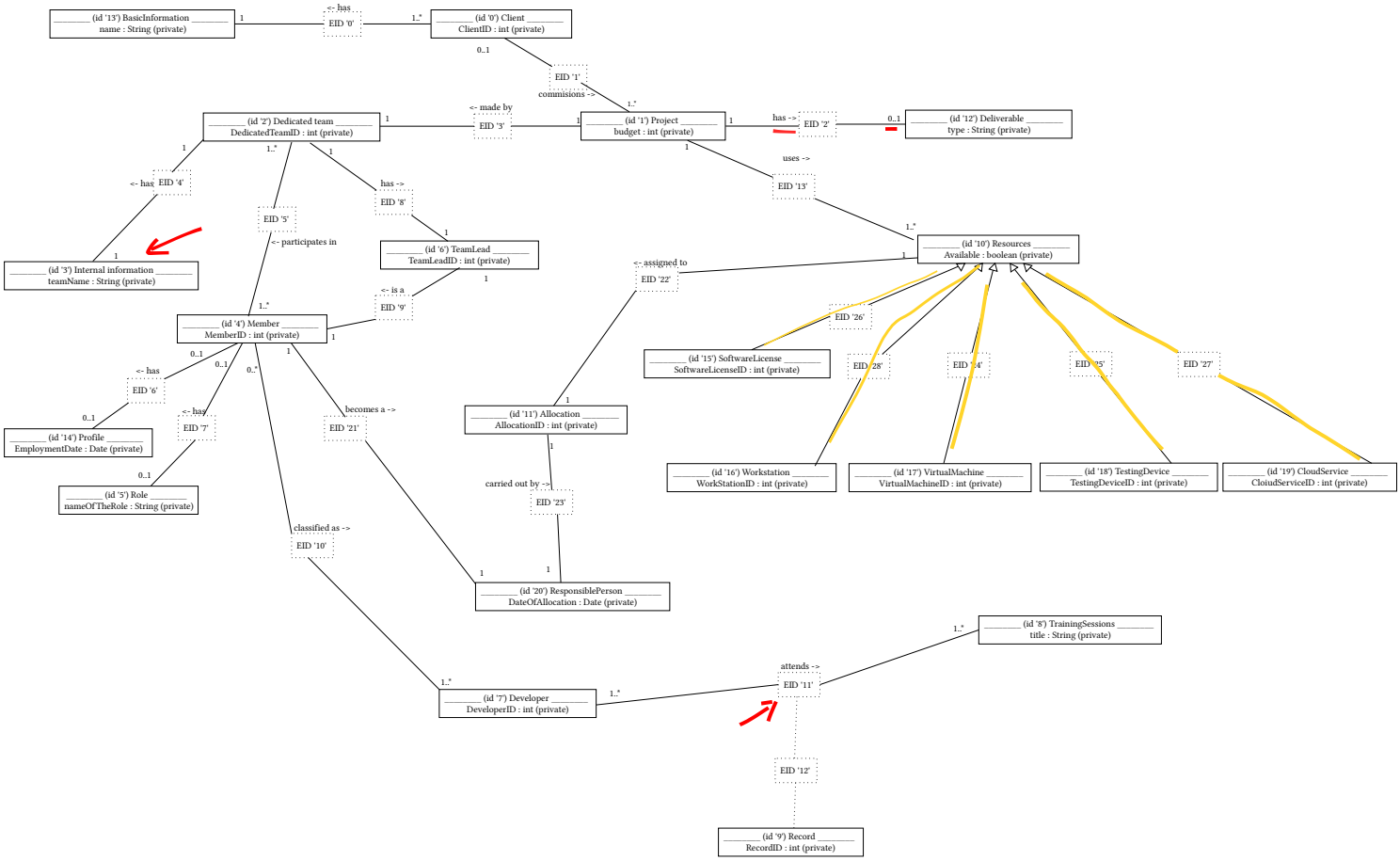

Figure 8: A GraphViz view of the corrected internal representation of submission 154286. Note that due to the limitations of GraphViz we show edges as two edges with a fake node inbetween. Edge paths are correctly stored internally.

4.3.3.1. Edge Label Swapping

Firstly, *Seshat* supports edge label swapping. It can happen during the diagram creation process that a student creates labels on an edge, but then drags the labels to other locations. This can be seen with the edge 'Project' → 'Deliverable', which has swapped labels 'has ->' and '0..1', in Figure 7 and Listing 1.

Having swapped labels is a problem for automatic grading, because *Seshat* does not consider the visual representation of the submission. When a student drags around labels and the diagram tooling does not automatically swap the labels internally, the file structure does not match the visual structure, which would make *Seshat* grade a diagram 'incorrectly' according to student and teacher expectations, putting students at an unfair disadvantage.

Seshat includes an option `swap_edge_labels` for this, which looks for labels with large offsets towards another position on the edge and swaps them if possible. The distance at which this occurs is configurable as a percentage of the length of an edge.

4.3.3.2. Edge 'Anchoring'

Some diagram software, such as UTML, sadly does not allow connecting edges to other edges, which is mandatory syntax for concepts such as UML association classes. Alternatively, students may drag arrows close to vertices or edges but not connect them directly for several possible reasons, including time limitations of exams.

This is a problem for autograders, since *technically*, those edges are not connected to anything, meaning that students will get points deducted for a solution that looks visually correct.

In order to allow some level of leniency which compensates for the inability of students to connect edges and/or the diagram software's inability to support it, *Seshat* allows for 'anchoring' disconnected edges that are sufficiently close to another vertex or edge. The distance at which anchoring occurs can be configured as a percentage of the length of an edge. for vertices, this is the length of the top, left, right, or bottom

part of the vertex (which is assumed to be a rectangle). A demonstration of this reparation can be seen in the connections 'Record' and 'Internal Information' in Figure 8.

4.3.3.3. Directed Edge Recombination

In diagrams, it is not uncommon to have a set of multiple vertices connect to one 'main' vertex. This happens especially often when drawing multiple inheritance classes in UML. However, diagram software might not make it easy to draw these in a visually pleasing manner, or students might choose to draw separate edges which capture the semantics visually, but which is not represented in the underlying diagram format. A good example of this is presented in Figure 7, where classes ranging from 'SoftwareLicense' until 'CloudService' connect to 'Resources'. However, these edges are drawn individually as straight edges, and any given edge does not connect an inheriting class to the inherited class.

This is a major problem for autograding, as the representation mismatch will cause autograders to grade the individual edges, instead of the semantic concept (for example, multiple-inheritance).

Seshat includes an option `simplify-directed-edges` for this, which looks for multiple edge-to-edge connections that end in a directed edge (an edge with some kind of arrow or diamond on one side). The effect of this recombination is demonstrated in Figure 8.

4.3.4. Grading process

After the internal representation is repaired, it can be graded. This is done based on graph comparisons / isomorphism. It requires (1) a teacher solution, (2) a student solution, and (3) a grading rubric which instructs the autograder which error corrections to apply and how many points to give or subtract.

Given a teacher and student graph, *Seshat* first attempts to map the vertices and edges of the teacher graph (reference) to those drawn by the student (submission). *Seshat* grades with the following general graph comparison algorithm:

1. take the teacher's reference graph (r) and the student submission graph (s). Vertices in a graph g are denoted by V_g. Edges in g are denoted by E_g.
2. analyse the semantic and syntactic equivalence of all $(v_r, v_s), v_r \in V_r, v_s \in V_s$ and score it in a range of 0 (no similarity) to 1 (perfect similarity), i.e. $score(v_r, v_s) \to [0..1]$.
3. for each $v_r \in V_r$, take $max(score(v_r, v_s))\forall v_s \in V_s$, given that the score is higher than the similarity threshold defined in the grading instructions. Put these pairs $v_r \to v_s$ into a minimal subgraph pair (g_{v_r}, g_{v_s})
4. repeat for each pair g_{v_r}, g_{v_s} until no progress is made:
   1. get a list of $e_r \in E_r$ that connect to $v_r \in g_{v_r}$ ($E_{g_{v_r}}$), and a list of $e_s \in E_s$ that connect to $v_s \in g_{v_s}$ ($E_{g_{v_s}}$).
   2. $\forall e_r \in E_r$ get the starting and ending vertices of e_r (if they exist), collect them into $V_{g_{v_r}}$. Do the same for E_s ($V_{g_{v_s}}$).
      - for each $v_r \in V_{g_{v_r}}$, make a mapping of $max(score(v_r, v_s))\forall v_s \in V_{g_{v_s}}$ and collect it into `newFixedIds`.
      - for each $e_r \in E_{g_{v_r}}$, try to match the first edge in $E_{g_{v_s}}$ for which the starting/ending vertices are in `newFixedIds`. Collect these into `newFixedEdgeIds`.
   3. Add all $(v_r, v_s) \in$ `newFixedIds` and add all $(e_r, e_s) \in$ `newFixedEdgeIds`.
5. Once no further progress is made, score the subgraphs $\left(g_{v_r}, g_{v_s}\right)$ based on how many vertices and edges have been mapped: $score\left(g_{v_r}, g_{v_s}\right) \to \mathbb{R}$.
6. Take the highest-scoring subgraph pair as basis.
   - For all other subgraphs pairs $g_{v_r}, g_{v_s})$, sorted by score, add , given that the edges and vertices of other subgraph pairs do not appear in the final combined solution yet.

After the last step, we have one final mapping $\left(g_{v_r}, g_{v_s}\right)_{\text{fin}}$ which decides which vertices of the reference solution likely map to the student submission.

After arriving on a final mapping, we apply the user-supplied grading configuration: we hand out additions or deductions in points based on vertex or edge presence, absence, or incorrect-

ness, and for specific parts of vertices or edges such as vertex attributes or edge arrow style.

This grading configuration also specifies the reparation options specified in Section 4.3.3. This allows a teacher / grader to specify stricter or looser corrections for a particular dataset, if *Seshat* performs undesired repairs.

4.3.5. Visualisation

After calculating a grade, *Seshat* needs a way to show this to the user. Three built-in methods exist for exporting data: export only the final grades to a `.csv` file, export the final grade and a detailed reasoning for why this is the final grade to a `.json` file per submission, or export both a `.csv` file and `.json` files. An example of a `.json` grade export can be seen in Listing 2.

We also include a demo graph export of a grade in GraphViz format, which uses the same graph export functionality as seen in Figure 8 to visualise a grade. It shows in red, yellow, and green which vertices / edges were incorrect, superfluous, or correct.

There also exists a GraphViz export function for *Seshat*'s internal structure. This allows for easily viewing how different reparation options affect a solution. A JSON export of the graph is also possible, which outputs the exact internal representation of the graph. An example is given in Figure 8. Note that GraphViz neither supports fixed edge paths nor edge-to-edge connections, meaning that edge placement is incorrect, however, this is present in the JSON export.

```
{ "final_grade": 4.95, "reason": {
  "missing_reference_vertices": {},
  "vertex_grades": { ... },
  "missing_reference_edges": {},
  "edge_grades": {
    "0": {
      "presence": { "grade": 1, ... "explanation": "present" },
      "line_style": { "grade": 0.25, ... "explanation": "equal to sample solution" },
      "vertex_start": { "grade": 0.1, ... "explanation": "equal to sample solution" },
       "vertex_end": { "grade": 0.1, ... "explanation": "equal to sample solution" }
    },
    "3": {
      "presence": { "grade": 1, ... "explanation": "present" },
      ...
      "vertex_start": { "grade": 0.1, ... "explanation": "equal to sample solution" },
      "vertex_end": { "grade": 0.05, ... "explanation": "50% correct multiplicity: should have been '1'" }
    }, ...
  }},
  "vertex_mapping": { "0": 0, "1": 1, ... },
  "edge_mapping": { "0": 0, "1": 1, "2": 2, "3": 3, "4": 4 }
}
```

Listing 2: Snippets from the `.json` grade export of submission 1027326 from TCS 2025 q.6

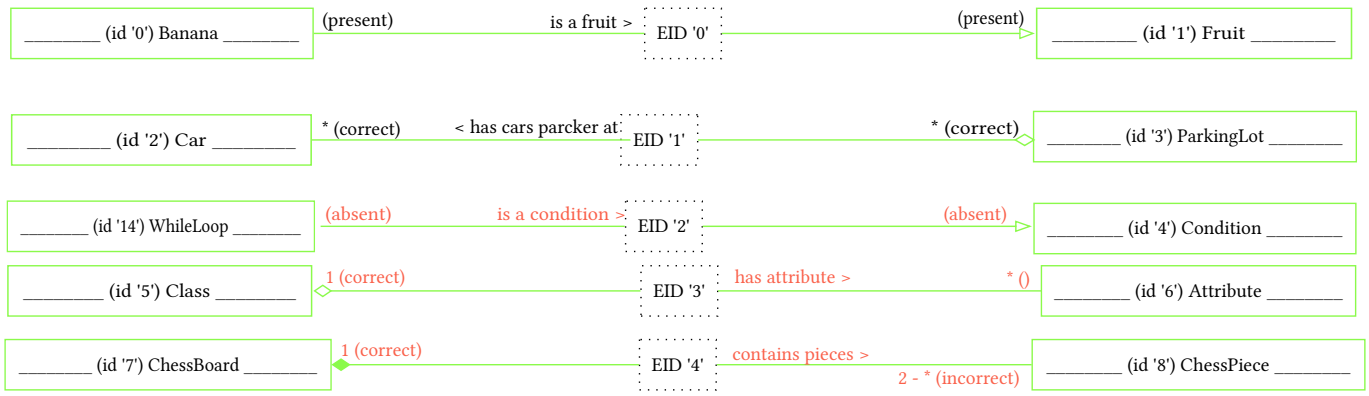

Figure 9: Demo graph (`.dot`) export of submission 1027326 from TCS 2025 q.6.

4.4. Testing

When developing an autograder, it is vitally important to verify its correctness and expected behaviour. We implement a variety of automated tests to ensure this. Because *Seshat* includes a lot of parsing and graph corrections, not unlike compilers and Syntax Tree / Control Flow Graph analysis, we take inspiration from the terms used for compiler testing, as introduced in V. Zaytsev [53].

Seshat is tested in an automated fashion for all large features it possesses: for parsing, conversion into the internal graph structure, repairing the internal graph, and for grading.

For the initial stage of parsing UTML into our own in-memory data structure, we employ P-testing [53]. This means that we verify that, for each file in our test data the data sets, *Seshat* should produce the exact same JSON structure as is inputted. One noteable exception is UTML's `attributes` and `methods` fields. the UTML files sometimes either lack these fields, they are `null`, or they contain an an empty array (`[]`). Because this is semantically equivalent in our context, we explicitly treat `null`/`[]` or a missing `attributes`/`methods` field as the same.

For the conversion from UTML into the internal representation, we use a form of N-testing [53]: we parse a UTML file into `ParseResultUTML`, then convert it into our `InternalGraph`, and then perform checks comparing the parse result and internal representation. We validate whether the vertex and edge IDs remain the same, and whether edges are still connected to their respective vertices or edges, to name a few.

For special features such as connecting edge ends and swapping labels (see Section 4.3.3) we perform unit testing with fixed examples, which test both a couple of happy paths (where the program should modify the graph) and control paths (where the program should not change the graph).

5. THREATS TO VALIDIDTY

The internal validity of this paper may be affected by a few factors.

Firstly, selection bias in the researched papers and reporting bias in those papers may affect the perceived performance of certain types of autograders, which could negatively affect the choice of algorithms used for *Seshat*. As we aim to provide a *comprehensive* overview, not an exhaustive one, selection bias becomes more important, especially when using snowballing, as one can easily fall into multiple papers about a specific sub-strategy and lose the overall picture. This is also why we keep snowballing at a minimum.

Secondly, the manner in which humans have constructed the rubrics in the dataset is inherently biased towards human graders, which generally do not apply it to the letter (which we will see in Section 6). When implementing a grading rubric that *does* follows the rubric to a tee, we risk losing the leniency of human interpretation.

Thirdly, the datasets do not provide details about which humans graded which submissions, when those submissions were graded, which reasoning was behind each of the grades, along with a variety of other factors. Especially the reasoning behind a grade would have been useful to discern the different types of grading deductions, which would help with result quality. However, since we only get a final grade to work with, we have to guess why grades were constructed by humans, and aim to achieve a similar leniency and error correction. This process is additionally prone to bias from the author.

6. RESULTS

For gathering results, we employed the following methodology: we read the rubric and, if present, the explanatory text of the exercise, and make a first sample solution. We also exactly construct a grader config with error repairs on and we award / subtract as many points as the grading rubric instructs to.

Then, we run *Seshat* on the dataset with our initial rubric, combine the human and autograding into one file, and graph the difference in grading in the paper. We aim to get a flat line, meaning that there is no difference in human or autograding.

We inspect the outliers, sorting by biggest difference and sampling roughly every 20 submissions. We note our initial results in the paper, giving the initial average difference. We determine if the difference is up to human error or if *Seshat* is configured or programmed wrongly. If *Seshat* is incorrect, we revise the code or the grading rubric and grade the entire dataset again.

We repeat this procedure two to five times, after which we write down our final results.

We save the raw scores, example rubrics, and the results in our code repository [48].

6.1. BIT 2024

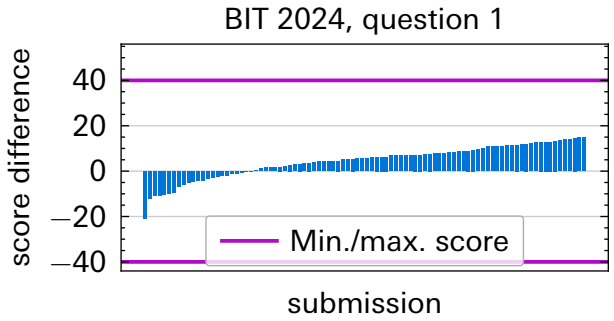

Figure 10: Difference in grades in the BIT 2024 dataset. Sorted by increasing difference.

The BIT 2024 dataset contains one question (q.1) about drawing a class diagram for an Electric Vehicle Charging Network. The exercise expects a simple UML class diagram, with the focus on the presence of certain classes and association types.

After implementing the rubric in code exactly, giving 1 point per present class and association, along with a point in total for correct edge multiplicities, the scores were extremely negative compared to the human grading, with average autograder scores being on average 10 points lower. After inspecting a few of the most outrageous offenders with differences of 20-30 points it turned out *Seshat* was not mapping edges correctly between the solution and submission. After resolving this, and giving *just over* 1 point for each present vertex and edge (simulating human forgiveness), the equivalence improved drastically with an average difference of 4.34 out of 40 points. This can be seen in Figure 10.

TODO - re-inspect and grade again max. three times and write results in final report. Don't forget to write down human/seshat errors!

6.2. TCS 2025

6.2.1. Question 5

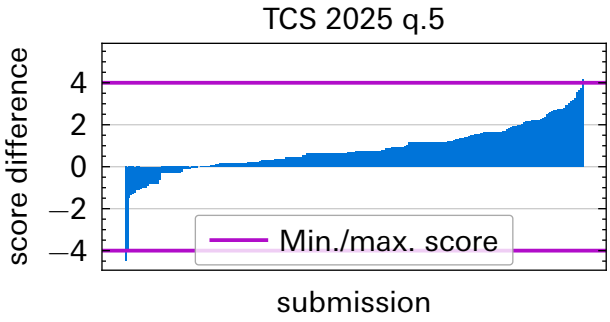

Figure 11: Difference in human and automatic grading for the TCS 2025 dataset, question 5. Sorted by increasing difference.

Question 5 asks to make a UML class diagram that models a ficticious theme park. The sample rubric is quite concise, giving one point for the correct classes, one point for correct methods / attributes, and a combined two points for correct associations and association types.

Here, the strategy we opted for was similar to the BIT 2024 dataset, giving only scores for present classes and associations and not deducting points for extra classes. Unlike Section 6.1, the scores awarded by *Seshat* were enormous at first, regularly reaching over 10 points higher than the TA grading, while the maximum score for the exercise was 5. After adjusting the grading of classes and associations to only award a point in *total*, and not *per element*, the grading looked a lot more reasonable, at an average grade difference of 0.82 out of 4 points.

However, there are still outliers which were graded differently by more than half the total score. TODO regrade two / three more times like BIT 2024.

6.2.2. Question 6

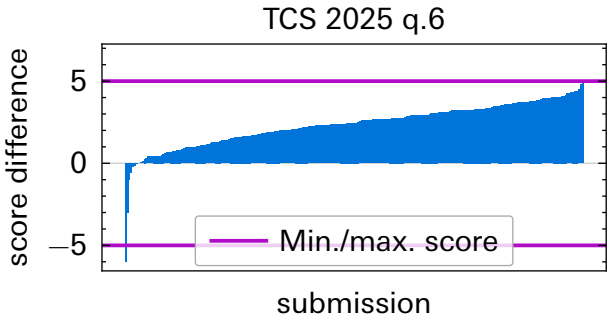

Figure 12: Difference in human and automatic grading for the TCS 2025 dataset, question 6. Sorted by increasing difference.

Question 6 is all about associations: it asks to draw the correct (types of) associations with the correct multiplicities between predetermined

classes. As a consequence, the original grading rubric and our grading configuration only awards points for correct associations and association types.

However, initially, *Seshat* gave quite optimistic scores, on average giving out scores that were 2.3 higher. To compensate,

6.3. BIT 2025

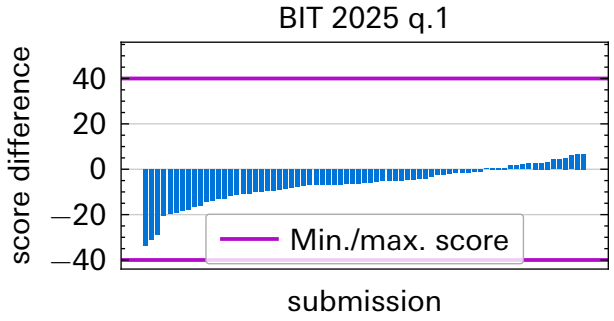

Figure 13: Difference in human and automatic grading for the TCS 2025 dataset, question 6. Sorted by increasing difference.

The BIT 2025 dataset asks students to make a relatively complicated UML class diagram that appears to model the relationships in software development teams. The rubric hands out individual points for present classes and for asscociations, as well as some points for specific multiplicities. The strategy to emulate this grading the best was to give points for present classes and associations that are mentioned in the rubric, as well as giving small fractions of points for correct multiplicities. After revising the grading a time or two, we arrived at an average difference of −6.62 of out 40 points. regrade 2-3 times like previous datasets - look at the lowest scoring solutions and see why it's wrong, likely some edge detection thing or multiplicities..

7. DISCUSSION

As seen in Section 6, *Seshat* can emulate human grading pretty effectively, with most differences stemming from conclude this in results (likely human error) - but verify and then put results here. However, it remains important to manually check a few solutions, especially the ones that receive a lower grade. From our investigations, *Seshat* can very effectively detect correct solutions, but it is sometimes harsh when grading solutions that are slightly different from the sample solution in subtle ways.

However, it takes a few rounds of revising the example solution and the scoring system to get an approximate grade, meaning that human grading is not very strictly following the rubric. This is partially due to *Seshat* not being able to find certain edges add examples but also partially due to the inherent inconsistency, or possibly foregiveness, in human grading.

More generally: autograding with a sample solution offers another paradigm of grading: instead of having a teacher make a rubric for a particular exercise, it forces them to think about possible solution graphs. The possible example solutions are the rubric. This works quite well for exercises with defined solutions (ex.: 2025 TCS question 6, 2025 BIT), but for less clear exercises, the number of possible solutions expands rapidly. This makes more ambiguous exercises inherently worse for autograding when using example solutions.

7.1. Future work

ILOs could have been added to the grading configuration, providing a nice feature, explain how, but there's not a lot of use when comparing pure scores. But it's nice for students to see how well they understand each learning objective.

Seshat is currently not very user-friendly, and very unfit for use by people that are not intimately familiar with the terminal, as *Seshat* currently only offers a terminal-based CLI. Given its architecture, it is relatively trivial to build a graphical interface around it, and it would be an interesting idea to see if *Seshat* can, with some user experience improvements, be adopted into the grading workflow, using the suggested workflow given in Section 6.

As mentioned in Section 7, *Seshat* does not directly work with the grading rubrics that teachers have been using traditionally. It might be interesting to modify *Seshat*'s grading process to be able to take in such a grading rubric, and see which style of grading is preferred by teachers. This could give more insights into the possible adoption of autograders by teaching staff.

8. CONCLUSION

In this research, we investigate to what extent we can automate the process of grading UML diagrams while maintaining or improving the *accuracy*, *consistency*, and *grading transparency* of human grading. To achieve this, we consult existing work for possible solutions, but find no implementations that share the full program (source code or other types of instructions) *and* that offer high accuracy, consistency, grading transparency, and support UTML or can be extended to support it.

Hence, we build our own autogarder: *Seshat*, which uses the best performing algorithms from existing work: a custom structural graph matching algorithm based on the work of P. Thomas *et al.* [27] to match parts of a sample solution to a given student submission, semantic matching using the sentence transformer `all-MiniLM-L6-v2` to account for synonyms, and syntactic matching using `Levenshtein` distance to account for spelling mistakes.

While building and testing *Seshat*, we discover that UTML fundamentally cannot express certain UML concepts and that student submissions often appear visually correct but have an incorrect internal structure. We implement, explain, and showcase a few *reparation* methods to still be able to grade 'imprecise' solutions.

Finally, we compare *Seshat*'s grading against human grades and discover that write results.

To answer our main research question: we can almost entirely automate the process of grading diagrams. The only thing a human needs to do in order to use *Seshat* is to provide a grading rubric specifying how many points to award/ deduct for certain attributes of a diagram and to provide a (set of) sample solution(s). However, to ensure the grading aligns with the expectations of a teacher, the results need to be randomly investigated, and the grading rubric or sample solution(s) possibly revised a few times.

It is also possible to automate this process while **improving** consistency, transparency, and fairness, compared to human grading. results todo.

word this nicely:

- now the challenge for teachers becomes to design unambiguous exercises and to encode grading rubrics into *Seshat*.
- *Seshat* offers a new way of viewing diagram grading: example solutions become the rubric, and the focus shifts to developing clearer exercises to minimise the number of possible solutions. Clear exercises are more easily automatable because they require less alternative graphs. This is a benefit to both students and teachers: teachers get a positive nudge towards developing exercises that are clearer for students and get rewarded with more accurate autograding and students get clearer exercises. (there is something to be said about disambiguating statements in communication, that is a nice skill, but it is not inherently related to diagram creation. I think it is good to separate the two, because the exercises are not labeled 'make diagram and communicate with stakeholders')

add more

BIBLIOGRAPHY

[1] Object Management Group, "OMG website." [Online]. Available: https://www.omg.org/

[2] F. Ahmed *et al.*, "Teaching Assistants as Assessors: An Experience Based Narrative," 2024. [Online]. Available: https://research.utwente.nl/files/457355611/126242.pdf

[3] M. Meadows *et al.*, "A Review Of THe Literature On Marking Reliability." [Online]. Available: https://assets.publishing.service.gov.uk/media/5a820a57e5274a2e87dc0d5a/0505_Meadows_and_Billington_CERP_RP.pdf

[4] D. Osinga, "Combining Dynamic and Static Analysis for the Automation of Grading Programming Exams," Bachelor thesis, 2024. [Online]. Available: https://github.com/osingaatje/ut-bachelor-thesis

[5] W. Bian *et al.*, "Is automated grading of models effective?: assessing automated grading of class diagrams," in *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems*, in MODELS '20. ACM, Oct. 2020, pp. 365–376. doi: 10.1145/3365438.3410944.

[6] I. G. Pirie, *The measurement of programming ability*. University of Glasgow (United Kingdom), 1975. [Online]. Available: https://search.proquest.com/openview/3afb41488c34283f38dec7f13a3ea744/1

[7] M. J. Rees, "Automatic assessment aids for Pascal programs," *ACM SIGPLAN Notices*, vol. 17, no. 10, pp. 33–42, Oct. 1982, doi: 10.1145/948086.948088.

[8] N. Smith *et al.*, "Interpreting imprecise diagrams," in *Diagrammatic Representation and Inference*, in International Conference on Theory and Application of Diagrams. Mar. 2004, pp. 239–241. doi: 10.1007/978-3-540-25931-2_24.

[9] P. Thomas, "Grading Diagrams Automatically," 2004. [Online]. Available: https://oro.open.ac.uk/90155/1/2004_01.pdf

[10] S. Bagui *et al.*, *Database design using entity-relationship diagrams*. Auerbach Publications, 2003. [Online]. Available: https://www.taylorfrancis.com/books/mono/10.1201/9780203486054/database-design-using-entity-relationship-diagrams-sikha-bagui-richard-earp

[11] OMG, "XML Metadata Interchange format." [Online]. Available: https://www.omg.org/spec/XMI

[12] IBM, "Rational Rose." [Online]. Available: https://www.ibm.com/support/pages/ibm-rational-rose-enterprise-7004-fix-pack-4-7000

[13] PlantUML, "PlantUML." [Online]. Available: https://plantuml.com/

[14] Visual Paradigm, "Visual Paradigm." [Online]. Available: https://www.visual-paradigm.com/

[15] D. Huistra, "UTML - internal GitLab repository." [Online]. Available: https://gitlab.utwente.nl/ewi/eduapps/UTML/

[16] University of Twente, "utml.apps.utwente.nl." [Online]. Available: https://utml.apps.utwente.nl/

[17] A. M. Soaita *et al.*, "A methodological quest for systematic literature mapping," *International Journal of Housing Policy*, vol. 20, no. 3, pp. 320–343, Aug. 2019, doi: 10.1080/19491247.2019.1649040.

[18] F. Batmaz, "Semi-Automatic Assessment ofStudents' Graph-Based Diagrams," 2010. [Online]. Available: https://www.academia.edu/download/66135135/70_22270_EM_26aug_20feb_L.pdf

[19] V. Vachharajani *et al.*, "A Proposed Architecture for Automated Assessment of Use Case Diagrams," in *International Journal of Computer Applications (0975 – 8887)*, 2014. [Online]. Available: https://www.academia.edu/download/67672696/pxc3900193.pdf

[20] W. Bian *et al.*, "Automated Grading of Class Diagrams," in *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, IEEE, Sept. 2019, pp. 700–709. doi: 10.1109/models-c.2019.00106.

[21] M. Hosseinibaghdadabadi *et al.*, "Automated Grading of Use Cases," in *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, IEEE, 2023. [Online]. Available: https://ieeexplore.ieee.org/iel7/10343461/10343549/10343598.pdf

[22] O. Anas *et al.*, "New method for summative evaluation of UML class diagrams based on graph similarities," 2021. [Online]. Available: https://www.academia.edu/download/66135135/70_22270_EM_26aug_20feb_L.pdf

[23] H. AlRawashdeh *et al.*, "Using Model Checking Approach for Grading the Semantics of UML Models," 2014. [Online]. Available: https://iieng.org/images/proceedings_pdf/8684E0114567.pdf

[24] M. Striewe *et al.*, "Automated Checks on UML Diagrams," in *ITiCSE'11*, in ITiCSE '11. ACM, June 2011, pp. 38–42. doi: 10.1145/1999747.1999761.

[25] P. Thomas *et al.*, "An approach to the automatic grading of imprecise diagrams," technical report, 2006. doi: .org/10.21954/ou.ro.00016046.

[26] P. Thomas *et al.*, "Automatically assessing graph-based diagrams," *Learning, Media and Technology*, vol. 33, no. 3, pp. 249–267, 2008, doi: 10.1080/17439880802324251.

[27] P. Thomas *et al.*, "Automatically Assessing Diagrams," in *Proceedings of the IADIS International Conference on e-Learning*, 2009. [Online]. Available: https://www.researchgate.net/profile/Pete-Thomas/publication/42799920_Automatically_assessing_diagrams/links/0fcfd5060076dd8ba2000000/Automatically-assessing-diagrams.pdf

[28] P. Thomas *et al.*, "Generalised Diagramming Tools with Automatic Marking," in *Innovation in Teaching and Learning in Information and Computer Sciences*, 2011. doi: 10.11120/ital.2011.10010022.

[29] N. Smith *et al.*, "Automatic Grading of Free-Form Diagrams with Label Hypernymy," in *2013 Learning and Teaching in Computing and Engineering*, IEEE, Mar. 2013, pp. 136–142. doi: 10.1109/latice.2013.33.

[30] D. R. Stikkolorum *et al.*, "Towards Automated Grading of UML Class Diagrams with Machine Learning," 2019. [Online]. Available: https://ceur-ws.org/Vol-2491/paper80.pdf

[31] C. Wang *et al.*, "Assessing UML Diagrams by GPT: Implications for Education," technical report, 2025. [Online]. Available: https://www.researchgate.net/publication/397720325_Assessing_UML_Diagrams_by_GPT_Implications_for_Education

[32] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," 2023. [Online]. Available: https://arxiv.org/abs/2201.11903

[33] R. Ranjan *et al.*, "A Comprehensive Survey of Bias in LLMs: Current Landscape and Future Directions," 2024. doi: 10.48550/arXiv.2409.16430.

[34] M. Brenndoerfer, "Why Temperature=0 Doesn't Guarantee Determinism in LLMs," 2025, [Online]. Available: https://mbrenndoerfer.com/writing/why-llms-are-not-deterministic

[35] B. Atil *et al.*, "Non-Determinism of "Deterministic" LLM Settings," 2025. [Online]. Available: https://arxiv.org/pdf/2408.04667

[36] N. Bouali *et al.*, "Toward Automated UML Diagram Assessment: Comparing LLM-Generated Scores with Teaching Assistants," 2025. [Online]. Available: https://research.utwente.nl/files/496461589/134819.pdf

[37] A. F. Ferraris *et al.*, "The architecture of language: Understanding the mechanics behind LLMs," *Cambridge Forum on AI: Law and Governance*, vol. 1, pp. 1–19, 2025, doi: 10.1017/cfl.2024.16.

[38] R. Ramachandran *et al.*, "AI Assisted System for Automated Evaluation of Entity-Relationship Diagram and Schema Diagram Using Large Language Models," technical report, Dec. 2025. doi: 10.3390/bdcc10010002.

[39] A. Dinur *et al.*, "Incorporating outcomes assessment and rubrics into case instruction," *Journal of Behavioral and Applied Management*, vol. 10, no. 2, pp. 291–311, 2009, [Online]. Available: https://jbam.scholasticahq.com/article/17258.pdf

[40] S. Modi *et al.*, "A Tool to Automate Student UML diagram Evaluation," 2021. [Online]. Available: https://www.academia.edu/download/72488756/575.pdf

[41] R. Jebli *et al.*, "Assessing Students' UML Class Diagrams: a NewAutomated Solution," in *2023 7th IEEE Congress on Information Science and Technology (CiSt)* |, IEEE, 2023. [Online]. Available: https://ieeexplore.ieee.org/iel7/10409867/10409868/10409936.pdf

[42] N. H. Ali *et al.*, "Assessment System For UML Class DiagramUsing Notations Extraction," 2007. [Online]. Available: https://www.researchgate.net/profile/Zarina-Shukur/publication/253243639_Assessment_System_For_UML_Class_Diagram_Using_Notations_Extraction/links/55487af30cf2b0cf7acec2e4/Assessment-System-For-UML-Class-Diagram-Using-Notations-Extraction.pdf

[43] N. H. Ali *et al.*, "A Design of an Assessment System for UML Class Diagram," in *2007 International Conference on Computational Science and its Applications (ICCSA 2007)*, IEEE, Aug. 2007, pp. 539–546. doi: 10.1109/iccsa.2007.2.

[44] S. Foss *et al.*, "Learning UML database design and modeling with AutoER," in *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, in MODELS '22. ACM, Oct. 2022, pp. 42–45. doi: 10.1145/3550356.3559091.

[45] S. Foss *et al.*, "Automatic Generation and Marking of UML Database Design Diagrams," in *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education*, in SIGCSE 2022. ACM, Feb. 2022, pp. 626–632. doi: 10.1145/3478431.3499376.

[46] S. Foss, "AutoER: A System for the Automatic Generation and Evaluation of UML Database Design Diagrams," 2022. [Online]. Available: https://open.library.ubc.ca/media/download/pdf/24/1.0421624/4

[47] M. D. Wilkinson, "The FAIR Guiding Principles for scientific datamanagement and stewardship," 2016. [Online]. Available: https://www.nature.com/articles/sdata201618

[48] D. Osinga, "Seshat." [Online]. Available: https://codeberg.org/drwr/seshat

[49] Hugging Face, "sentence-transformers/all-MiniLM-L6-v2." [Online]. Available: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[50] Princeton University, "WordNet - A Lexical Database for English." [Online]. Available: https://wordnet.princeton.edu/

[51] Rust Team, "Overview of the Rust Compiler." [Online]. Available: https://rustc-dev-guide.rust-lang.org/overview.html

[52] Google, "The Go Programming Language." [Online]. Available: https://go.dev/

[53] V. Zaytsev, "An Industrial Case Study in Compiler Testing (Tool Demo)," 2018. doi: 10.1145/3276604.3276619.

APPENDICES

A GRADING RUBRICS

TODO Add rubrics from datasets 2024 BIT, 2025 TCS q5+q6, 2025 BIT